20 10 2026 Royel Iván Jima Pardo

Del AVR ⇝ ESP-32

Activación

```
DDRB |= (1<< PB5);
PORTB |= (1<< PB5);
if( PIND & (1<< PD2));
```

⚠ 4 cables
SPI → Comunicación entre la Flash y CPU 6 = 1 SPI

PortB → 8 pines

	ATmega328P	ESP32-WROOM
Ancho de Palabra	8 bits	32 bits
Nucleos/reloj	1 nucleo · 16 MHz	2 núcleos Xtensa LX6 hasta 240 MHz
SRAM	2 KB	520 KB
Flash	32KB, dentro del chip	4 MB, chip aparte en el modulo (SPI)
Tensión lógica	5 V	3,3 V ⚠
Pin ↔ periferico	fijo (PB5 = bit 5 PORTB)	configurable (GPIO matrix)

↳ Registro

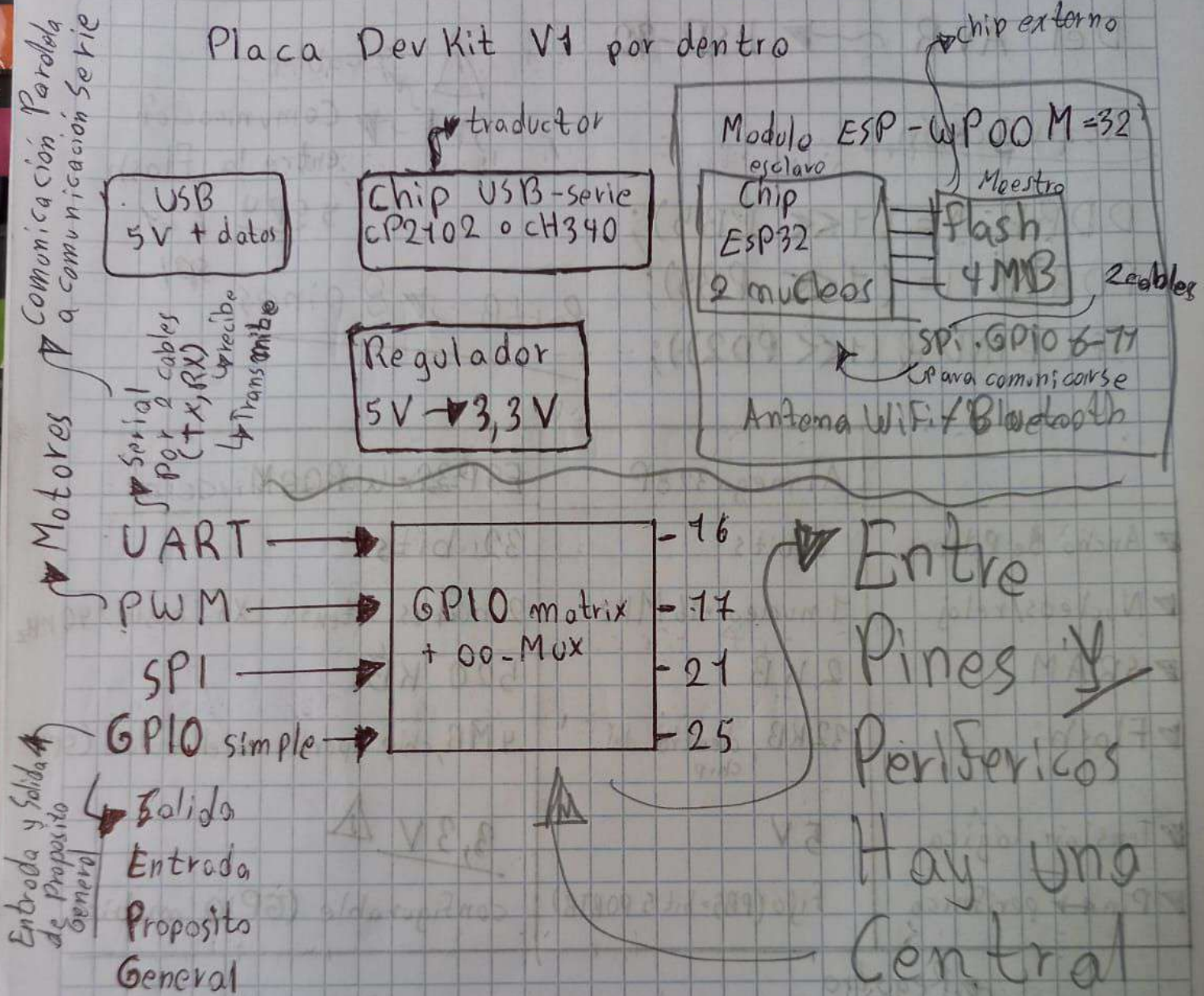
⚠ Un pin se controla escribiendo bits en registros mapeados en memorica

↳ flip flops
↳ Guarda 1 bit

Periferico ⇝ Circuito interno
↳ función especifica
↳ liberar CPU

DÍA MES AÑO NOTA

Placa Dev Kit V1 por dentro



Función	AVR	ESP32 (GPIO 0-31)
Dirección (int/out)	DDRB	GPIO_ENABLE_REG
Escribir la salida	PORTB	GPIO_OUT_W1TS_REG
Leer la entrada	PINB	GPIO_IN_REG
Pull-up interna	PORT (pin como entrada)	

DÍA 21 MES 10 AÑO 2026 NOTA /

Riesgo de leer, Modificar y escribir

↳ RMW (Read - Modify - Write)

↳ ESP-32

↳ 2 Nucleos

↳ Ambos Nucleos ejecutan instrucciones en paralelo

Si ambos nucleos leen y modifican el mismo registro al mismo tiempo ↳ El riesgo de sobrescribir los cambios del otro núcleo se multiplica drásticamente

⚠ Solución

↳ Registros de escritura atómico

↳ ESP-32



↳ Disponen de registros atómicos

W1TS	VS	W1TC
Write 1 to set		Write 1 to clear
• Bits en 1 ↳ encienden		• Bits en 1 ↳ se apaga
• Bits en 0 ↳ no se toca		• Bits en 0 ↳ no se toca

↳ GPIO_OUT_W1TS_REG ~ Write 1 to set

GPIO_OUT_W1TC_REG ~ Write 1 to Clear

DÍA MES AÑO NOTA /

Capas de abstracción

Más fácil

1. tu codigo ~> digitalWrite(16, HIGH);
2. Core Arduino-ESP32 ~> llama a ESP-IDF
3. ESP-IDF (Espressif) ~> gpio_set_level(16, 1);
4. Registro ~> GPIO_OUT_W1TS_REG |= 1<<16
5. Hardware ~> el pin 16 sube a 3,3V

↳ más control y velocidad

Mapa de pines del DevKit V1 (30 pines)

Strapping - lee el chip los lee al arrancar

VP -> Positivo

VN -> Negativo

UART0 ~> carga y Monitor Serie

Vin 5 Voltios ~> Pin sin regular

3V3 ~> sacar 3.3voltios

GND ~> tierra

GND ~> tierra

Vin ~> 5 voltios

↳ Encender la ESP-32

ESP-IDF → librerios con las que vamos a trabajar

DÍA 27 MES 10 AÑO 2026 NOTA

Strapping → Bootloader ROM

GPIO0 (cero)
↳ en bajo al arrancar modo de carga

3,3 V → limites electricos



LR → resistencia de carga
↳ Cualquier aparato que necesito corriente para funcionar
Se transforma en Energia

Led → lo quema el Voltaje
↳ A más voltaje más corriente

⚠ señal de control desde el ESP-32

Bootloader ROM — lee el programa
2ª bootloader (flash) — cargara el programa
app_main — ESP-IDF librerias a trabajar
loopTask — loop — Se cargan las variables globales
loop — se ejecuta pa siempre

int8_t ↳ -128..127 / uint8_t 0...255 → Desbordamiento se reinicia de nuevo

int16_t
-32768...32767

Volatil
↳ Puede cambiar sin que se vea / Static

Antirebote
↳ Hasta que se estabilice se lee

Semana 3

DÍA 28 MES 10 AÑO 2026 NOTA /10

Temporizadores de hardware en el ESP32

GPIO → 34 → 39 → Entrada

Variable
- Global → se reinicia en cada loop
- local
- Static

Promoción
- Cantidad de bits a utilizar

Temporizador → Periferico
- Circuito dentro del microcontrolador que cuenta pulsos de reloj
  - Como cada pulso dura un tiempo conocido
    - sirve para medir tiempo
    - Avisar cuando ha pasado un tiempo determinado.



Temporizador
- timer

Hardware → Cristal de Cuarzo

frecuencia 60 Hz Ecuador

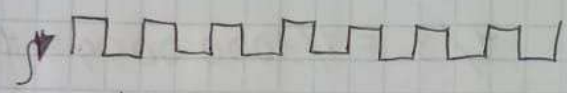
- delay (1000)
  - se bloquea hasta que se termine el tiempo
- millis () → CPU lee
  - un loop donde revisa el tiempo
    - en los registros
- Temporizador
  - circuito aparte
    - avisa la hora exacta

Contador → Suma 1 en cada pulso
- Registro del temporizador → Circuito Sumador pegado
  - espacios de la memoria
- la CPU / cuando lo inicia / cuando inicia / cuando termina

$RAM → Variables que van cambiando

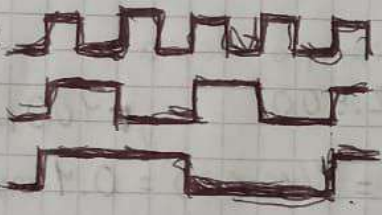
28 10 2026

CPU → 1. configura
↳ 2. Habilita
↳ 4. lo detiene
3. Temporizador cuenta solo / 80 MHz ↳ 12,5 nano segundos



Reloj → Señal que sube y baja a ritmo fijo, como un metrónomo. cada subida es un pulso

APB → 80 millones de pulsos por segundo → 80 MHz
↳ Bus de periféricos del ESP32 ↳ cada pulso ↳ 12,5 nanosegundo

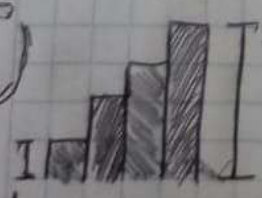
Prescaler → Divisor antes del contador
↳ Ejemplo:  → Menos preciso ↳ memoria
↳ divisor que deja pasar al contador solo 1 de cada N pulsos de reloj
→ Más preciso ↳ más memoria

Resolución → 1/f → pulsos tick por segundo
↳ duración de un tick  ↳ Duración de cada pulso
↳ tiempo más pequeño que se puede distinguir

Frecuencia
↳ Cantidad de pulsos en un determinado tiempo

Tiempo = alarma x resolución
dos pájaros = tiempo x frecuencia

Alarma → Número que seguarda en un registro
→ Alarma Cuando el contador lo alcanza Temporizador genera un aviso.
↳ Nosotros la elegimos 55
↳ se guarda en un registro



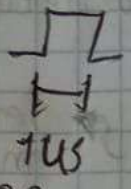
↳ tick

{ Registro contador +1 = Registro Alarma 55

Ejemplo: f=1MHz

Datos: f=1000 000 ticks por segundo ; tiempo deseado = 0,5s

① Resolución 1/f = 1/1000 000s = 1µs



② Alarma t.f = 0,5s x 1000 000 = 500 000

③ Comprabación 500 000 x 1µs = 0,5s

④ Prescaler N = 80 000 000 / 1 000 000 = 80

f = 10.000 ; tiempo deseado

2.500 ↓ 100µs

① Resolución 1/f = 1/10.000 = 0.1ms

② Alarma t.f = 2500 x 10000 = 0.25s

③ Comprabación ...

① Prescaler N = 80000 000 / 10.000 → N = 8000

Autorrecarga → Temporizador vuelve a 0 cada vez que se cumple la alarma

↳ Poca Memoria pero impreciso

Desbordamiento

DÍA MES AÑO NOTA

04 11 2026

Interrupciones Externas

Sondeo (polling) → Preguntar una y otra vez en loop si algo ya pos

Interrupción → Aviso del hardware, pausa el programa para atender algo urgente → Alarma

Interrupción externa → cuyo aviso que llega desde un pin → con un boton

ISR → Interrupt Service Routine
↳ función que se ejecuta al llegar el aviso

Flanco →  → instante en que una señal cambia de 0 a 1 o de 1 a 0

Rebote → Cambios falsos y rapidísimo

Compilador →

Registro CPU → Casillas muy rapidas dentro de la CPU donde trabaja con copias de datos

Caché → Copia pequeña y rápida de lo que la CPU leyó hace un poco de la flash

Watchdog → Perro guardian.
↳ reinicia el chip si algo se queda trabado mucho tiempo

DIA MES AÑO NOTA 7

Sondeo

↳ loop() lee el pin en cada vuelta



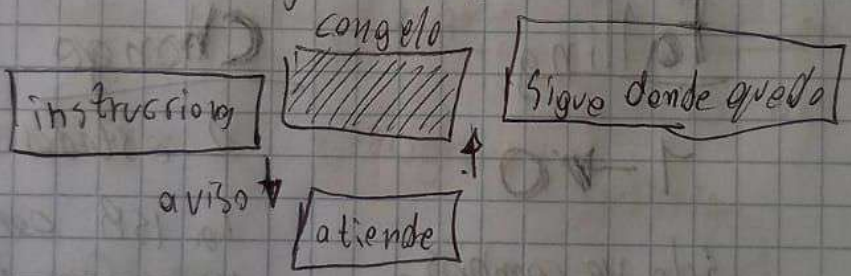
↳ trabaja con resistencias internas

↳ Pull UP → siempre envia un 1

↳ hasta que se aplaste el boton, donde envia un 0 logico

Interropción

↳ el aviso llego solo



① Pauso el loop en la instrucción que iba

② Ejecuta la ISR completa

③ Regresa a la instrucción siguiente donde quede

¿Quien vigila el pin?



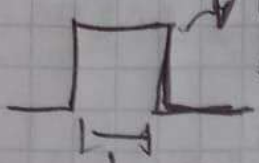
→ se presiona

↳ Pin GPIO21 → Periferico GPIO → CPU

1 → 0 detecta el cambio en hardware

pausa el loop y corre la ISR

Nivel frente a flanco



instante de cambio de estado
Flanco

Nivel que dura

Poll UP interna en ESP-32 → siempre esta conectada a 3,3 V ≈ 1 logico

Rising

0 → 1

Solo ve cambios de 0 a 1

Nota 1 cambio

Falling

1 → 0

Solo ve cambios de 1 a 0

CHange

2 estados

La ISR corre 2 veces por toque en cuanto los dos cambios

Nota 2 cambios



0 → 1

DÍA MES AÑO NOTA

attachInterrupt()

función

attachInterrupt(digitalPinInterrupt(21), isrBoton, FALLING);

¿Vigila?

digitalPinToInterrupt(21)

21 → pase de 1 a 0 llamar a la función

ISR → función → Interrupt Service Routine
↳ para hacer algo
↳ la activamos → Interrumpe algo para
↳ la llama el hardware hacer otr cosa

⚠ Pero no demorar mucho time

Donde vive el programa

DRAM ← Dirige CPU ← Cache
copia de lo leido hace poco

IRAM → IRAM_ATTR
↳ codigo casi de emergencia

• Guarda instrucciones

Compilador



loop | ocupado | sigue ocupado | atiende

ISR

bandera